# 06 — Retrieval-Augmented Generation

*How to give a language model the right information at the right moment, and how to measure every step of doing so.*

*Reading time: about 45 minutes. You need: modules 00 to 05; module 00's description of attention helps with one comparison. You do not need: any experience with search engines or vector databases.*

---

## A question the system cannot answer

This module's exercise builds a retrieval system over this course's own ten articles, split into 223 passages. Here is a real query from it:

> why is there a square root of $d_k$ in the attention score?

The answer is in module 01. But the three best passages found by keyword search come from modules 02, 00 and 09, and the three found by meaning-based search come from 00, 07 and 02. None of the six contains the answer. The first passage that does appears at roughly position 25 for keyword search, and position 11 for meaning-based search.

Now imagine handing those six passages to a language model and asking it to answer. However capable the model is, it cannot answer from text that does not contain the answer. It can only guess, or recall something from its training.

That is the central fact of **retrieval-augmented generation**, or RAG: retrieval quality sets the ceiling on answer quality. This article treats RAG as a pipeline of separate steps, each of which can be measured, and fixed, on its own.

## What you will learn

By the end of this article, you will be able to:

- describe the stages of a RAG pipeline, and what goes wrong at each;
- explain how keyword and meaning-based retrieval work, and why they fail differently;
- measure retrieval with recall, MRR and nDCG;
- combine retrievers, choose an index, and rerank results;
- check citations automatically, so that unsupported claims are caught.

Section 1 lays out the pipeline. Sections 2 to 8 follow it from raw text to ranked results. Section 9 covers generation and citations. A recap and the exercise briefing follow at the end.

---

## 1. The pipeline

A RAG system answers a question in several stages:

```
documents -> chunking -> indexing -> retrieval -> reranking
          -> context assembly -> generation -> citation
```

In words: split the documents into passages, and build a searchable index. Then find candidate passages for the question, and put the best ones in order. Finally, give them to the language model, and check that its answer is supported by what it was given.

The weak answer to "how would you build RAG?" describes only the happy path. It says: embed the documents, store them in a vector database, and put the top five into the prompt. The strong answer knows, for each stage, what can go wrong, how to measure it, and the cheapest fix.

### A connection to attention

Attention and retrieval do the same basic job. Attention compares a query with every key and takes a weighted average of the values. Retrieval compares a question with every passage and takes the best few. The differences are instructive:

| | Attention | Retrieval |
|---|---|---|
| memory searched | the tokens in the context window | an external index of documents |
| how items are chosen | a softmax over every key | the top few results from an index |
| similarity measure | learned during training | fixed embeddings, or keyword scores |
| trained end to end? | yes | usually not |
| how it is updated | by retraining the model | by rebuilding the index |

---

## 2. Chunking: deciding what a passage is

**Chunking** splits documents into the passages that retrieval returns. It is easy to overlook, and mistakes here cannot be fixed by later stages.

There are two common approaches:

- Fixed-size chunks cut the text every few hundred characters or tokens, usually with some **overlap** between neighbours. The overlap matters because an answer that crosses a boundary would otherwise be split in two.
- Structure-aware chunks follow the document's own structure: headings, paragraphs or sentences.

The difference shows up immediately. Here is the start of one fixed-width chunk from the exercise:

```
it will need. Calling .backward() walks that graph in reverse topological ...
```

It begins in the middle of a sentence, and so do 157 of the 225 fixed-width chunks. Almost no sentence-aware chunks do. The difference affects retrieval: with sentence-aware chunks, a relevant passage appeared in the top ten for 92% of the test questions, against 75% with fixed-width chunks.

### How big should a chunk be?

Chunks that are too small lose context: a passage saying "it reduces memory by 8 times" is useless without knowing what "it" is. Chunks that are too large mix several topics, so a single embedding represents none of them well. The exercise tried five sizes, using keyword search:

| Chunk size (characters) | Chunks | Relevant passage in top 10 | MRR |
|---|---|---|---|
| 200 | 787 | 67% | 0.507 |
| 400 | 408 | 67% | 0.465 |
| 700 | 223 | 92% | 0.606 |
| 1,200 | 134 | 92% | 0.536 |

| Chunk size (characters) | Chunks | Relevant passage in top 10 | MRR |
|---|---|---|---|
| 2,000 | 79 | 92% | 0.678 |

The small sizes clearly hurt. Beyond 700 characters, the top-ten figure stopped changing, and MRR moved up and down without a clear pattern. With only 12 test questions, those differences are too small to trust. The honest conclusion is that very small chunks fail here, and choosing among the larger sizes needs a bigger evaluation set.

Two more practical points follow. First, clean the text before chunking. In the opening example, the top keyword result was a line from a reference list, which happened to share words with the query. Second, **contextual retrieval** helps a lot in practice: before indexing, add a sentence to each chunk describing where it sits in its document.

A useful test for any chunking scheme: give a person only the retrieved chunk. If they cannot answer the question from it, the problem is chunking, not the model.

---

## 3. Keyword retrieval: BM25

**BM25** scores a passage by the query words it contains, with three corrections to simple word counting:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\frac{|d|}{\mathrm{avgdl}}\right)}$$

Here $f(t,d)$ counts how often word $t$ appears in passage $d$, $|d|$ is the passage length, and avgdl is the average length. The usual settings are $k_1 = 1.2$ and $b = 0.75$. IDF, the inverse document frequency, measures how rare a word is across the $N$ passages, $n$ of which contain it:

$$\mathrm{IDF}(t) = \ln\left(1 + \frac{N - n + 0.5}{n + 0.5}\right)$$

Each correction is easy to see on a tiny example:

- **Rare words count more.** In a three-passage test corpus, "the" appeared in two passages, so its weight is $\ln(1 + 1.5/2.5) = 0.470$. "cat" appeared in one, giving $\ln(1 + 2.5/1.5) = 0.981$.
- **Repetition saturates.** A passage repeating "cat" 50 times scored only 1.28 times as high as a passage containing it once. The $k_1$ term makes each extra occurrence count for less.
- **Long passages are penalised.** The $b$ term divides by relative length, so padding a passage with filler words lowers its score. Setting $b = 0$ switches this off.

BM25's strength is exact matching: product codes, error messages, names, numbers and rare jargon. Its weakness is vocabulary mismatch. A query about "cars" does not match a passage about "automobiles".

---

## 4. Meaning-based retrieval: bi-encoders

**Dense retrieval** turns the query and each passage into a vector, called an **embedding**, and ranks passages by how similar their vectors are. The model that produces the vectors is called a **bi-encoder**, because it encodes the query and the passages separately.

That separation is the key design point. Passage embeddings can be computed once, in advance, and stored in an index. Answering a query then costs one embedding plus a search, which is what makes dense retrieval scale to billions of passages.

The separation is also the limit. Each passage's vector is computed without knowing the question, so one vector must summarise everything the passage could be asked about.

Bi-encoders are trained with a **contrastive** objective: pull a question's vector towards the vector of a passage that answers it, and push it away from other passages. The most valuable training examples are **hard negatives**: passages that look relevant but are not.

The exercise builds its dense retriever without any download. It uses latent semantic analysis: a table of word counts, compressed with the SVD from module 01. It is a real dense retriever, but a weak one.

### The two families fail in different places

Here is the MRR of each retriever on four of the test questions. MRR is 1.0 when the first result is relevant, and 0.5 when the first relevant result is second:

| Question | BM25 | Dense |
|---|---|---|
| what is the pipeline bubble and how do I shrink it? | 1.00 | 0.50 |
| how do I check that my backward pass is correct? | 0.11 | 0.04 |
| the model runs but the loss does not go down, what do I check first? | 0.50 | 1.00 |
| what is the difference between a view and a copy in pytorch? | 0.25 | 0.50 |

Keyword search did better where a distinctive word such as "bubble" appears in both the question and the answer. Dense retrieval did better on the third question, whose answer talks about "overfitting a batch" rather than repeating the question's words.

---

## 5. Measuring retrieval

You cannot improve what you do not measure. Four measures cover most needs:

- **Hit@k** asks whether at least one relevant passage appears in the top $k$. It is the ceiling on what the language model can do.
- **Recall@k** is the fraction of all relevant passages that appear in the top $k$.
- **MRR**, mean reciprocal rank, averages $1/\text{rank}$ of the first relevant result. A relevant first result scores 1; a relevant third result scores $1/3$.
- **nDCG@k** handles graded relevance. Each result earns $2^{\text{relevance}} - 1$ points, divided by $\log_2$ of one more than its rank, and the total is divided by the best possible total.

nDCG deserves a worked example. Take three passages with relevance grades 3, 1 and 1. In the ideal order the three terms are $7/\log_2 2 = 7$, $1/\log_2 3 = 0.631$ and $1/\log_2 4 = 0.5$, adding up to 8.131, and nDCG is 1

by definition. Reverse the order and the same terms become 1, 0.631 and 3.5, adding up to 5.131. So nDCG is $5.131/8.131 = 0.631$, because the best passage now sits in the least valuable position.

On the course corpus, the exercise's retrievers score as follows:

| Retriever | Hit@10 | Recall@10 | MRR | nDCG@10 | Recall@50 |
|---|---|---|---|---|---|
| BM25 | 0.917 | 0.647 | 0.606 | 0.505 | 0.858 |
| dense (LSA) | 0.833 | 0.612 | 0.636 | 0.494 | 0.935 |
| hybrid (RRF) | 0.833 | 0.585 | 0.609 | 0.486 | 0.893 |
| hybrid + reranker | 0.917 | 0.626 | 0.632 | 0.498 | 0.858 |

The discipline matters more than the choice of measure. Build an evaluation set of real questions, with their relevant passages labelled, before you tune anything. Without one, every change is a guess.

---

## 6. Combining retrievers

Since keyword and dense retrieval fail in different places, it is natural to combine them. Their scores cannot be added directly: BM25 scores have no upper limit, while similarity scores lie between −1 and 1. The standard trick uses ranks instead. **Reciprocal rank fusion**, or RRF, gives each passage:

$$\mathrm{RRF}(d) = \sum_{\text{retrievers } r} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60$$

It rewards agreement. Suppose one retriever ranks passages 9, 5, 1 and the other ranks 7, 5, 2. Passage 5 is second in both, and RRF puts it first, above passages that only one retriever liked.

It is not a simple average of ranks, though. With rankings 1, 2, 3 and 3, 2, 1, the outer passages score $1/61 + 1/63 = 0.032266$ while the middle one scores $2/62 = 0.032258$. So a passage ranked first by one retriever and last by the other edges out one ranked second by both, by eight parts in a million. It is worth knowing this before relying on it.

### An honest result

On this corpus, combining the retrievers did not beat BM25 alone in the top ten, as the table in section 5 shows. The dense retriever is weak, and mixing a strong retriever with a weak one can cost precision even while it adds recall. Dense retrieval did find the most relevant passages within the top 50.

So "hybrid search is better" is a hypothesis to test on your own data, not a fact to assume. With a strong modern embedding model, fusion usually does help. The only way to know is to measure.

---

## 7. Indexes: exact and approximate

Searching every stored vector for every query is called exact search. It costs time in proportion to the number of passages times the vector length. That is perfectly fine up to a few hundred thousand passages, which is worth remembering before building something complicated. Beyond that, systems use **approximate nearest-neighbour** search.

**IVF** (inverted file index) groups the vectors into clusters with k-means. A query is compared with the cluster centres, and then only with the vectors in the closest few clusters. The number of clusters searched, called `n_probe`, sets the trade-off directly. Here it is for the course corpus, with 16 clusters:

| Clusters searched | Top-10 results that match exact search | Passages scanned |
|---|---|---|
| 1 | 28% | 17 |
| 2 | 43% | 29 |
| 4 | 67% | 59 |
| 8 | 86% | 115 |
| 16 | 100% | 223 |

Two other index types are common. **HNSW** builds a layered graph that connects each vector to its near neighbours; it gives excellent recall at high speed but uses more memory. **Product quantisation** compresses vectors into short codes, which makes searching billions of vectors in memory feasible.

Whatever the index, report its recall against exact search alongside its speed. A system that is 50 times faster but finds only 70% of the true top results may be much worse than one that is 10 times faster and finds 98%.

---

## 8. Reranking

The first stage of retrieval aims for recall: get the right passage somewhere among, say, the top 50. A second stage, **reranking**, aims for precision: put the best passages at the very top.

A reranker is usually a **cross-encoder**. Unlike a bi-encoder, it reads the question and a passage together, so every word of the question can interact with every word of the passage. That makes it far more accurate. It is also far more expensive: one model call per candidate passage, instead of one quick vector comparison. So it can only be applied to a short list.

The exercise cannot download a cross-encoder, so it uses a hand-written reranker that combines word overlap, exact phrase matches and how close the matching words sit. Even this simple reranker helps: nDCG rose from 0.486 to 0.498, and MRR from 0.609 to 0.632.

There is also a clear lesson in the table from section 5. Every first-stage retriever found a relevant passage within its top 50 for every test question. So on this corpus, the remaining problem is ranking, not retrieval, and that tells you where effort will pay off.

> **In the interview:** describe RAG as two stages: cheap retrieval for recall, then expensive reranking for precision. Then say how you would measure each stage separately.

---

## 9. Generation, grounding and citations

Retrieval can be measured precisely. Generation is where users decide whether to trust the system.

**Assemble the context carefully.** Language models attend unevenly across a long context, and tend to miss information in the middle, an effect known as "lost in the middle". So place the strongest passages first and last. Remove near-duplicate passages too: they waste space and suggest agreement between sources that does not exist.

**Ground the answer.** Instruct the model to answer only from the provided passages, and to say when they do not contain the answer. The failure to guard against is a fluent answer drawn from the model's training data instead. It looks exactly like a grounded answer unless you check.

**Verify every citation.** Ask the model to cite a passage identifier for each claim, then check each citation automatically. The exercise implements a simple check: what fraction of a sentence's informative words appear in the passage it cites. On a real example, a sentence taken from the cited passage scored 1.0. A made-up sentence, claiming the KV cache "is stored on a separate optical coprocessor", scored 0.33 against the same passage, below the threshold of 0.55, and was flagged.

That check has a clear limit. A false sentence that reuses the passage's own words, but reverses their meaning, would pass. Production systems therefore use a trained model that judges whether one sentence follows from another, a task called natural language inference.

**Know the failure types.** When a RAG answer is wrong, the cause is almost always one of these:

| What happened | Where to fix it |
|---|---|
| the answer was never retrieved | chunking or the retriever |
| it was retrieved, but ranked too low to be used | reranking |
| it was in the context, but the model ignored it | context order, or the prompt |
| the model answered from memory instead | grounding instructions and citation checks |
| the model refused, although the answer was there | an over-cautious prompt |

Evaluation tools such as RAGAS name the matching measures: *faithfulness* (is every claim supported?), *answer relevance* (does it answer the question?), and *context recall* (did retrieval find what was needed?).

---

## Recap

- Retrieval quality is the ceiling on answer quality. Measure each stage of the pipeline separately.
- Chunk along the document's structure, clean the text first, and avoid very small chunks.
- BM25 matches exact words; dense retrieval matches meaning. They fail in different places.
- Build an evaluation set before tuning, and measure with hit@k, recall, MRR and nDCG.
- Fusion and approximate indexes are trade-offs: measure their effect rather than assuming it.
- Rerank a short list for precision, and verify every citation automatically.

---

### *Retrieval measures, for reference*

Every measure used in the article, in one place. Each is averaged over the questions in the evaluation set, and each runs from 0 to 1, with higher being better. $k$ is how many results you look at, so Recall@10 judges the top ten. One warning about that letter: section 6 also calls the constant inside reciprocal rank fusion $k$, following the paper, and that $k$ is 60 rather than a cut-off.

| Measure | In full | What it counts | A worked value |
|---|---|---|---|
| Hit@$k$ | hit rate at $k$ | whether *any* relevant passage made the top $k$ | 1 if one did, else 0 |
| Recall@$k$ | recall at $k$ | the fraction of *all* relevant passages in the top $k$ | 2 of 4 found gives 0.5 |

| Measure | In full | What it counts | A worked value |
|---|---|---|---|
| MRR | mean reciprocal rank | how high the *first* relevant result sits | first place 1, third place 1/3 |
| nDCG@$k$ | normalised discounted cumulative gain | the whole ranking, when relevance has grades | see below |
| RRF | reciprocal rank fusion | not a measure: a way to merge two rankings | $\sum 1/(60 + \text{rank})$ |

The letters of nDCG name its three steps. The *gain* of a result with relevance grade $g$ is $2^g - 1$, so a grade of 3 is worth 7 and a grade of 1 is worth 1. That gain is *discounted* by $\log_2(\text{rank} + 1)$, which is what makes a high position worth more. The discounted gains are *cumulated*, and the total is *normalised* by the best total the same grades could have achieved, so a perfect ranking scores exactly 1.

Which to reach for depends on the question being asked:

- *Hit@k* when the generator needs only one good passage. It is the ceiling on what the language model can do, and the first thing to check when answers are wrong.
- *Recall@k* when an answer must draw on several passages, or when $k$ is the size of a shortlist handed to a slower reranker.
- *MRR* when position matters and relevance is yes-or-no. It is sensitive to the top of the ranking and ignores everything below the first hit.
- *nDCG@k* when passages are graded rather than merely relevant. It is the most informative of the four, and the only one that needs graded labels to be worth using.

A measure is only as good as the evaluation set under it. The figures in section 5 come from 12 questions, which is too few to separate close results, and the article says so where it reports them.

---

## The exercise: a measured RAG pipeline over this course

**What you will build.** A complete retrieval pipeline over this course's own articles, with no downloads. You write BM25, a dense retriever, the four evaluation measures, rank fusion, an IVF index, a reranker and a citation checker. You also compare chunking strategies. Because the corpus is this course, you can judge a bad result on sight.

**Time and setup.** Allow four to five hours. The setup is the same as for module 00. The corpus and the test questions come from `corpus.py` in the module folder.

**How to run it.** Open `06-rag-systems/exercise.ipynb`, or run `python exercise.py` from inside the module folder.

**How the checks work.** They work as in module 00. The first cell describes the corpus and shows a few test questions:

```
223 chunks from 10 articles, mean 619 chars
12 evaluation queries with graded relevance judgements

  why is there a square root of d_k in the attention score?
     -> 01-math-foundations, needs one of ('saturat', '1/\\sqrt{d_k}', ...
```

Each part then reports that it is not implemented, until you write it.

**Step map.** This table links each part to the section of the article it practises.

| Part | You write | The idea it practises | Section |
|---|---|---|---|
| 1 | `BM25` | keyword retrieval | 3 |
| 2 | `LSARetriever` | dense retrieval | 4 |
| 3 | `hit_at_k`, `recall_at_k`, `mrr`, `dcg`, `ndcg_at_k` | measuring retrieval | 5 |
| 4 | `rrf` | combining retrievers | 6 |
| 5 | (comparison code is provided) | chunking strategy and size | 2 |
| 6 | `IVFIndex` | approximate search | 7 |
| 7 | `rerank` | reranking | 8 |
| 8 | `supported`, `verify_answer` | citation checking | 9 |

**When you are done.** Your retrievers reproduce a table like the one in section 5. Your reranker improves nDCG and MRR over the stage it reranks. Your citation checker accepts the grounded sentence and flags the fabricated one.

Several parts end with a written question. One of the best is 8b: name a false claim your citation checker would not catch, and say what you would use instead.

**If you get stuck.**

- Check each measure against the small hand examples in the notebook before using it on the corpus.
- If BM25 gives strange results, print the IDF of a few common and rare words.
- When a query goes wrong, print the text of its top three passages. The cause is usually visible.
- If you are still stuck, `solution.py` has a complete reference.

**Want more?** The notebook ends with four stretch goals: make retrieval differentiable, add query expansion, add contextual retrieval, and implement the search procedure of an HNSW index.

---

## Going deeper

### Start here

- Gao et al. (2023), *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997. The best single map of RAG designs and their trade-offs. Read it after this article.

### Retrieval

- Robertson & Zaragoza (2009), *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval 3(4). The standard reference on BM25. Section 3 derives the saturation and length corrections.
- Karpukhin et al. (2020), *Dense Passage Retrieval for Open-Domain Question Answering*, EMNLP. arXiv:2004.04906. How to train a bi-encoder, including the role of hard negatives.
- Cormack, Clarke & Büttcher (2009), *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR. Two pages; the formula from section 6.
- Nogueira & Cho (2019), *Passage Re-ranking with BERT*. arXiv:1901.04085. The result that established cross-encoder reranking.
- Khattab & Zaharia (2020), *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*, SIGIR. arXiv:2004.12832. A middle ground between bi-encoders and cross-encoders.

- Deerwester et al. (1990), *Indexing by Latent Semantic Analysis*, Journal of the American Society for Information Science 41(6). The method behind the exercise's dense retriever.

**Indexes**

- Johnson, Douze & Jégou (2017), *Billion-scale similarity search with GPUs*. arXiv:1702.08734. The FAISS library, which implements IVF and product quantisation.
- Malkov & Yashunin (2016), *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. arXiv:1603.09320. The HNSW index.
- Jégou, Douze & Schmid (2011), *Product Quantization for Nearest Neighbor Search*, IEEE TPAMI 33(1).

**RAG systems and evaluation**

- Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS. arXiv:2005.11401. The paper that introduced the term.
- Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts*, TACL. arXiv:2307.03172. The evidence behind the context-ordering advice in section 9.
- Es et al. (2023), *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217.
- Anthropic (2024), *Introducing Contextual Retrieval* — https://www.anthropic.com/news/contextual-retrieval. The chunk-context technique from section 2, with measurements.
- Järvelin & Kekäläinen (2002), *Cumulated Gain-Based Evaluation of IR Techniques*, ACM Transactions on Information Systems 20(4). The original definition of nDCG.

---

**Next:** module 07 is about reading research papers critically, using FlashAttention as the example.